ML på ECE GPU CLUSTER

(Note[CEF]: Dokumentation lavet af SWMAL gruppe, efterår 2025. Publicering til andre SWMAL studerende godkendt af gruppen)

ML algoritmer bliver hurtigt meget krævende i forhold til processorkraft og hukommelse. De store neurale netværk kan desuden drage fordel af muligheden for parallel processering på nogle GPU'er. Til gengæld sidder den almene bruger sjældent med den dyre hardware der kan håndtere de helt store algoritmer…..inden for en overskuelig tidshorisont. Når der udvikles og ikke mindst trænes større modeller, foregår det derfor ofte på dedikeret hardware som server clusters (GPU clusters), så selve maskinen man sidder ved stadig kan bruges.

På AU er der adgang til et sådant GPU-cluster. En oversigt findes på denne adresse:
http://gpucluster.st.lab.au.dk/

Her findes dels en grafisk oversigt, links til man-pages og information om status på clusteret. Adgangen til clusteret kræver enten at brugeren er koblet på Eduroam eller koblet på via VPN/au-access med Cisco klienten:

https://studerende.au.dk/it-support/vpn

Login til clusteret forgår via Secure Shell (ssh) med gruppenavnet fra ML kurset. For eksempel, for gruppe 9, efterår i 2025 er det:

Login: *swmal09e25*

Password: *swmal09e25_123*

Når der logges ind på clusteret, er det i første omgang til "mastermode", hvorfra der så kan logges videre ind på node 6 – 13, der er beregnet til de tunge opgaver.

Jupyter notebook

Opgaverne i ML kurset foregår primært Jupyter notebooks der kører i et browser vindue. Her kan python koden afvikles i mindre blokke, der gør det nemt at undersøge funktionaliteten af individuel kode. Derudover kan der også lægges blokke med tekst (markup language) til beskrivelse osv.

På ovenstående link til clusteret, findes en række links til notebooks på de nævnte nodes. Der logges ind med credentials som nævnt ovenfor. I notebook'ens venstre side ses en filbrowser, hvor der bla. ligger filen

”demo_welcome.ipynb”, som vist på billedet herunder. Dobbeltklik på filen for at køre notebooken, og få den vist i den store rude til højre.

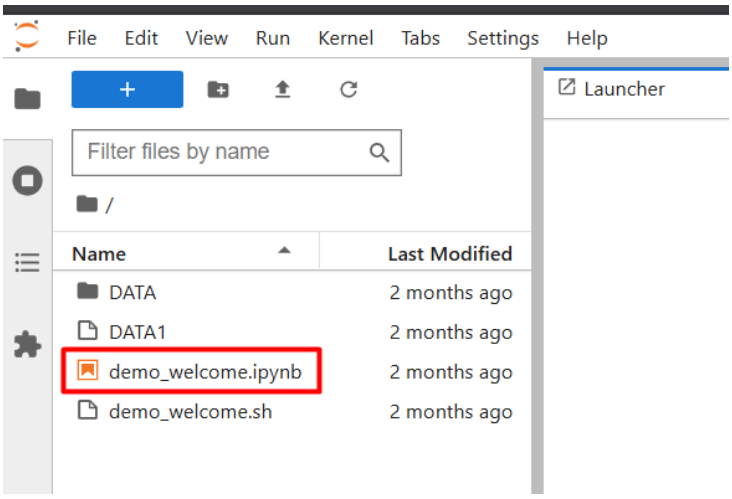

Figure 1

De individuelle kode-blokke afvikles ved at klikke på blokken og derefter ”play” knappen for oven (eller trykke [SHIFT]+[ENTER]).

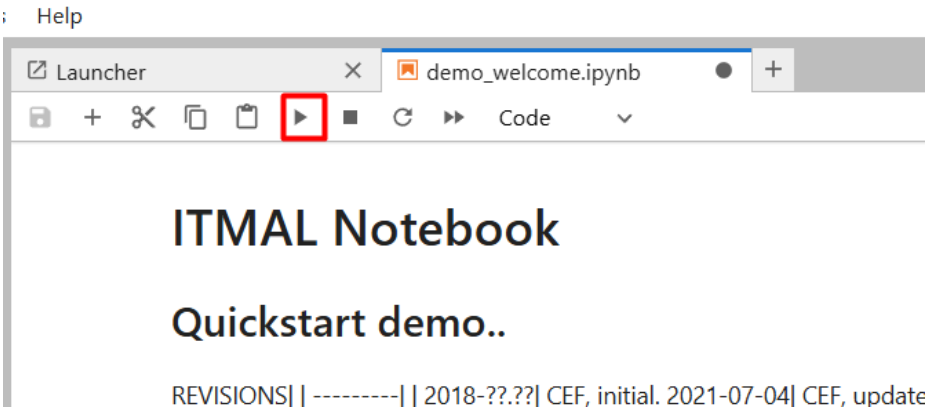

Figure 2

Den åbnede notebook indeholder bla. koden til at hente repositoriet til kurset GITMAL.

Koden der således køres i disse notebooks afvikles altså på clusterests nodes. Bemærk at der er en timeout på browseradgangen. Hvis der afvikles en algoritme der tager længere tid, tabes forbindelsen og afviklingen af koden afbrydes.

Terminal adgang

Tager afviklingen lang tid, er det derfor en fordel at ”afhænde” dette til serveren, så opgaven kan køre færdig uden at forbindelsen fra brugerens maskinen behøves opretholdt. Det kræver at koden ligger samlet på ”normal” vis, i stedet for som en notebook. Den lavpraktiske måde at gøre dette på er via Secure Shell (ssh). I det følgende bruges ”command promt” i windows.....pick your poison. Adressen er den samme som til oversigten, som det illustreres herunder:

```
Microsoft Windows [Version 10.0.19045.6456]
(c) Microsoft Corporation. Alle rettigheder forbeholdes.

C:\Users\soje>ssh swmal01e25@gpucluster.st.lab.au.dk
== The ECE GPU Cluster ==
(swmal01e25@gpucluster.st.lab.au.dk) Password:
       .---.        ____________
      /     \  /  < gpucluster >
      \.@-@./       ------------
      /`\_/`\
     //  _  \\
    | \     )|_
   /`\_`>  <_/ \
   \__/'---'\__/

MAINTAINER: cef@ece.au.dk

  System load: 0.01              Memory usage: 44%   Processes:       246
  Usage of /:  66.3% of 83.41GB  Swap usage:   0%    Users logged in: 2

STATUS[ 13:35  13/11-2025 ]:
  NODE4  : UP    (SLURM node)
  NODE5  : UP    (SLURM node)
  NODE6  : UP
```

Figure 3

Ved login sender serveren en mængde tekst om status osv. I bunden noters dette:

```
NOTICE: your are on the GPU cluster/MASTER node.
        Do not use this for computations,
        ssh to a computenode instead

FOUND no user keys in '/home/swmal01e25/.ssh/authorized_keys'
  You should consider using a password-less login
  See http://gpucluster.st.lab.au.dk/man7/gpucluster-ssh.html
swmal01e25@gpucluster:~$
```

Figure 4

Der er to informationer her. Den første er at vi nu er logget ind på ”mastermode”. Her kan vi arbejde med filhåndtering og skrive kode. Når vi vil *afvikle* koden, skal vi til gengæld logge videre ind på et node, da disse er designet til de tunge opgaver. Bemærk at filsystemet er fælles for

clusteret. Den kode der skrives, mens man er logget ind på mastermode, ligger også på nodes.

Den anden information er, at det kan blive omstændeligt at skulle logge ind med brugernavn og adgangskode 2 gange, hver gang vi skal foretages os noget fornuftigt. Vi kan slippe for at skulle give credentials ved hvert login, ved at oprette ”authorization-keys”. Det er et krypteret nøgle-fil sæt, hvor den ene ligger lokalt, og den anden lægges op på serveren.

Den lokale nøgle placeres i mappen ”.ssh” under hjemmemappen for brugerkontoen på maskinen. På følgende billede illustreres (på en windows maskine) hvordan man:

1. Flytter stien til brugerkontoens hjemme-mappe
2. Opretter mappen ”.ssh” (hvis den ikke allerede eksisterer)
3. Flytter stien til den oprettede mappe

```
C:\>cd %HOMEPATH%

C:\Users\soje>mkdir .ssh

C:\Users\soje>cd .ssh
```

Figure 5

Herefter oprettes nøglesættet med kommandoen vist i næste figur. Der spørges efter filnavn og passphrase. Hvis der bare trykkes [enter] alle tre gange, oprettes filerne med navnene:

id_ed25519

id_ed25519.pub

hvor filen med endelsen *.pub er filen der skal lægges op på serveren. Desuden undgår man at skulle indtaste password.

```
C:\Users\soje\.ssh>ssh-keygen -t ed25519 -C "din@email_adresse.dk"
Generating public/private ed25519 key pair.
Enter file in which to save the key (C:\Users\soje/.ssh/id_ed25519):
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in C:\Users\soje/.ssh/id_ed25519
Your public key has been saved in C:\Users\soje/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:81zZ+gfkqGpy8ZxoqPm3bZBMXhe4uV8AmwGKVaW0YrQ din@email_adresse.dk
The key's randomart image is:
+--[ED25519 256]--+
|      o.+o..     |
|     + + o+ .    |
|    . E o  B .   |
|     . .. * oo.  |
|       +So oo=.  |
|        *+....+  |
|       . *o+.. . |
|     .o *.* ..  .|
|    oo.*o+.   .. |
+----[SHA256]-----+

C:\Users\soje\.ssh>
```

Figure 6

Der er forskellige måder at kopiere *.pub filen op på serveren, hvor den tilsvarende skal ligge i ".ssh" mappen i bruger biblioteket. Den letteste metode er at bruge følgende kommando (secure copy) [13], der kopierer filen op hvor den skal ligge.

```
C:\Users\soje\.ssh>scp id_ed25519.pub swmal01e25@gpucluster.st.lab.au.dk:~/.ssh/authorized_keys
The authenticity of host 'gpucluster.st.lab.au.dk (10.28.27.23)' can't be established.
ED25519 key fingerprint is SHA256:G48NO8L4pD0BmrovfjeTV2x0N9IxoTUD6kVO+GWNd84.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])?
Warning: Permanently added 'gpucluster.st.lab.au.dk' (ED25519) to the list of known hosts.
== The ECE GPU Cluster ==
(swmal01e25@gpucluster.st.lab.au.dk) Password:
id_ed25519.pub

C:\Users\soje\.ssh>
```

Figure 7

Når der spørges om man vil fortsætte, indtastes "yes" og derefter indtastes adgangskoden (forhåbentlig for sidste gang). Herefter klares adgangskoden i baggrunden uden indtastning fra bruger.

Der skal stadig angives brugernavn som vist indtil nu (altså: brugernavn@server). Det kan dog også automatiseres. I ".ssh" mappen (på den lokale maskine) oprettes en ny fil med navnet "config" (altså uden fil-endelse!). I filen indtastes følgende to blokke.

```
Host cluster
  HostName gpucluster.st.lab.au.dk
  User swmal01e25
  IdentityFile ~/.ssh/id_ed25519
  IdentitiesOnly yes

Host node9
  User swmal01e25
  IdentityFile ~/.ssh/id_ed25519
  IdentitiesOnly yes
  ProxyCommand ssh -W %h:%p cluster
```

Figure 8

Første blok angiver brugernavn, nøgle og adresse på serveren, samt giver den et ”tag” der kan bruges i ssh kommandoen (her ”cluster”) i stedet for server adressen.

Anden blok gør næsten det sammen, men sidste linje sørger for, at vi kun skal køre kommandoen til at logge på node (altså ikke logge ind 2 gange). Bemærk at der skal laves en blok til hvert node man ønsker at bruge på denne måde.

```
C:\Users\soje>ssh cluster
== The ECE GPU Cluster ==
        .---.        ____________
       /     \  /   < gpucluster >
       \.@-@./       ------------
       /`\ /`\
```

Figure 9

```
C:\Users\soje>ssh node9
== The ECE GPU Cluster ==

 _______
< node9 >
 -------
   \
    \
        .--.
       |o o |
```

Figure 10

I Jupyter notebooks køres med en ”kernel” der forhåbentlig har de nødvendige biblioteker installeret. Til de scripts der skal kunne afvikles på en node, selv om man logger af, er det en god ide at arbejde i et Virtual Environment (det er det sådan set generelt). Her kan installeres biblioteker efter lyst, uden at det rører ved noget uden for den mappe man opretter miljøet i. I billedet herunder sker følgende:

1. Der oprettes en ny mappe med kommandoen *mkdir*
2. Stien flyttes ind i mappen med *cd*
3. Der oprettes et nyt virtuelt miljø
4. Indholdet af mappen vises med *ls*
5. Det virtuelle miljø aktiveres
6. Det virtuelle miljø deaktiveres

```
swmal01e25@node9:~$ mkdir projekt
swmal01e25@node9:~$ cd projekt
swmal01e25@node9:~/projekt$ python -m venv .venv
swmal01e25@node9:~/projekt$ ls -a
.  ..  .venv
swmal01e25@node9:~/projekt$ source .venv/bin/activate
(.venv) swmal01e25@node9:~/projekt$ deactivate
swmal01e25@node9:~/projekt$
```

Figure 11

For at lade en proces køre, selv om der logges af maskinen, kan den startes på følgende måde, hvor

- *nohup*: sørger for processen fortsætter med at køre
- *> output.log 2>&1 &*: gemmer output (inkl. fejlbeskeder) til en fil

```
(.venv) swmal01e25@node9:~/projekt$ nohup python -u noget.py > output.log 2>&1 &
[1] 1283149
(.venv) swmal01e25@node9:~/projekt$ pgrep -f noget.py
1283149
(.venv) swmal01e25@node9:~/projekt$ kill 1283149
(.venv) swmal01e25@node9:~/projekt$ cat output.log
nohup: ignoring input
noget
noget
noget
noget
noget
noget
```

Figure 12

Bemærk at processens ID vises når scriptet køres. Kan man ikke lige huske ID'et, kan man få det vist med *pgrep* som illustreret. ID'et bruges med kill kommandoen til at stoppe processen, hvis den ikke stopper af sig selv. *Cat* kommandoen bruges til at vise indholdet af logfilen.

VSCode

Ønsker man en editor med lidt (mange) flere features end *nano* eller *vim*, der ofter ligger på linux, kan VSCode stadig bruges her, da den har et

værktøj til at redigere filer remote. Nede i venstre hjørne trykkes på ikonet med de to pile/vinkler

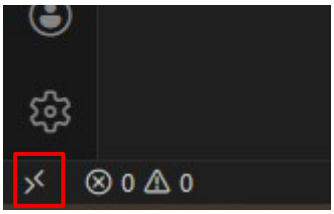

Figure 13

I menuen der åbnes for oven (indholdet kan variere efter installerede extensions) vælges ”Connect to Host…”.

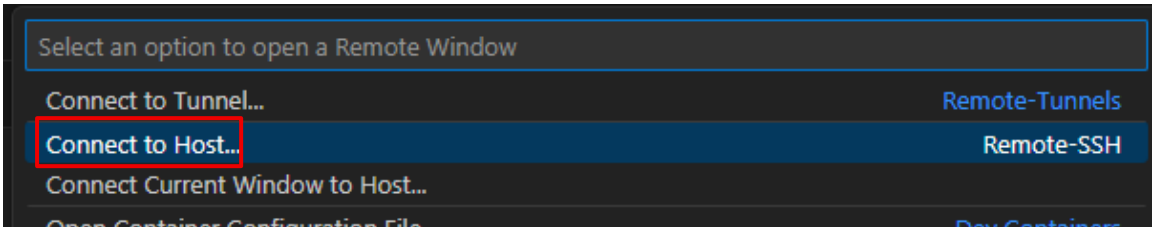

Figure 14

Herefter kan man vælge mellem de maskiner man har lagt i config filen nævnt tidligere.

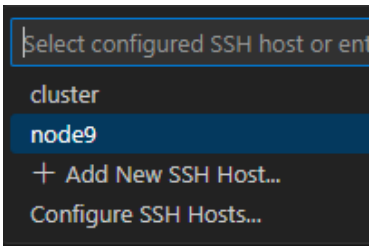

Figure 15

Herefter åbnes en ny rude med VSCode. Nede i venstre hjørne ses serveren der tilkobles.

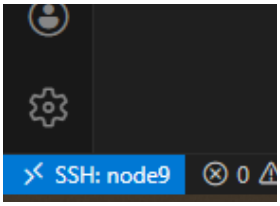

Figure 16

Ved første login bliver der spurgt om platformen på serveren – her vælges linux.

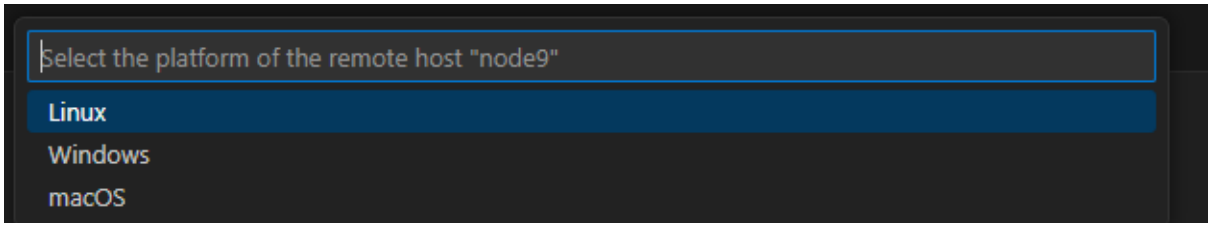

Figure 17

Herfra kan man arbejde, som man arbejder med filer på sin egen maskine. Trykker man på ”Open Folder…” får man vist filstrukturen på serveren, og man kan åbne det bibliotek man vi arbejde i. Åbner man en terminal igennem vscode, er den ligeledes koblet op på serveren.

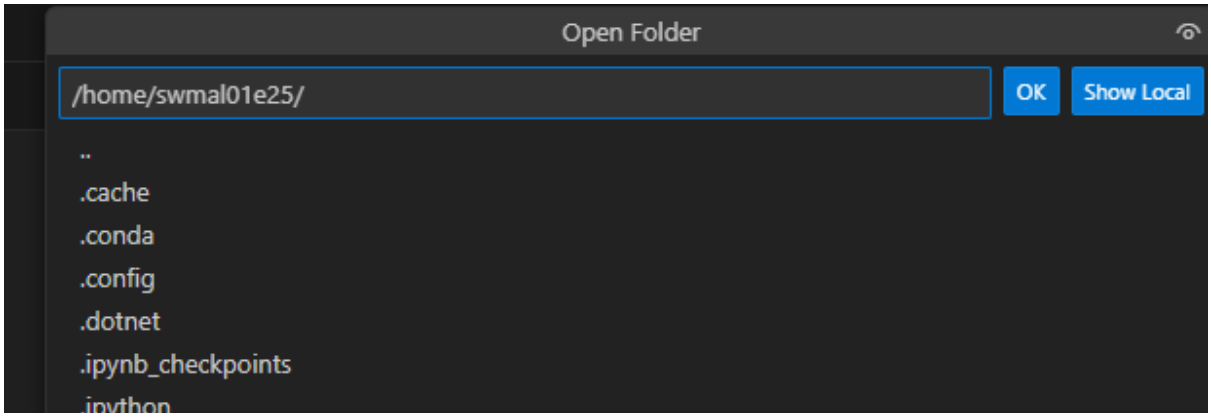

Figure 18

VSCode har i øvrigt en extension til Jupyter notebooks, så det hele kan håndteres i samme miljø.

Referencer

[1] A. Géron, Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow, O'Reilly, 2022.

[2] Tensorflow, »tfds.load,« [Online]. Available: https://www.tensorflow.org/datasets/api_docs/python/tfds/load. [Senest hentet eller vist den 13 11 2025].

[3] Tensorflow, »Better performance with the tf.data AP,« [Online]. Available: https://www.tensorflow.org/guide/data_performance. [Senest hentet eller vist den 13 11 2025].

[4] Keras, »Keras layers API,« [Online]. Available: https://keras.io/api/layers/. [Senest hentet eller vist den 13 11 2025].

[5] Keras, »Model training APIs,« [Online]. Available: https://keras.io/api/models/model_training_apis/#compile-method. [Senest hentet eller vist den 13 11 2025].

[6] pandas, »pandas.DataFrame.plot,« [Online]. Available: https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.plot.html. [Senest hentet eller vist den 13 11 2025].

[7] Keras, »Layer weight initializers,« [Online]. Available: https://keras.io/api/layers/initializers/. [Senest hentet eller vist den 13 11 2025].

[8] n_iter_no_change, »Scikit-Learn.org,« [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.GradientBoostingClassifier.html.

[9] add_dummy_feature, »Scikit-Learn,« [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.add_dummy_feature.html.

[10] ravel, »GeeksForGeeks,« [Online]. Available: https://www.geeksforgeeks.org/python/numpy-ravel-python/.

[11] max_iter, »Scikit-Learn,« [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html.

[12] tol, »Scikit-Learn,« [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html.

[13] J. Hayden, »SCP Linux – Securely Copy Files Using SCP examples,« [Online]. Available: https://linuxblog.io/linux-securely-copy-files-using-scp/. [Senest hentet eller vist den 13 11 2025].